Zurich University of Applied Sciences
School of Management and Law

Department Banking, Finance, Insurance

Machine Learning HS 25

Fraud Detection

Submitted by

Francis Du
Max Heinzer
Dominik Kaufmann

Supervised by

Bledar Fazlija

Winterthur, January 22nd, 2026

Management Summary

Credit card fraud detection presents a challenging machine learning problem due to extreme class imbalance and evolving fraud behavior. While synthetic oversampling techniques such as the Synthetic Minority Over-sampling Technique (SMOTE) are frequently used to address imbalance, their effectiveness under realistic deployment conditions remains unclear. This study examines whether applying SMOTE to the training data improves fraud-detection performance compared to training on the original imbalanced data.

Two supervised learning models, Random Forest and Support Vector Machine (SVM), are evaluated using a large credit card transaction dataset. Model development is conducted on historical data, while final performance is assessed on a temporally separated test set to ensure a realistic out-of-sample evaluation. All preprocessing, feature engineering, and model selection steps are confined to the training data to prevent information leakage. Decision thresholds are tuned on an internal validation set to reflect the higher operational cost of missed fraudulent transactions.

The results show that SMOTE-based oversampling does not improve fraud-detection performance for either model. For the Random Forest classifier, SMOTE leads to a substantial and statistically significant decline in fraud recall and overall performance. For the SVM, SMOTE produces an extreme trade-off, achieving very high recall at the cost of an impractically large number of false positives. In contrast, models trained on the original imbalanced data achieve better generalization when combined with careful threshold selection.

Table of Contents

List of Tables

List of Figures

1 Introduction

1.1 Background

Credit card fraud detection is a critical problem in modern financial systems, characterized by significant economic consequences and challenging data structures. The extreme rarity of fraudulent transactions relative to legitimate ones introduces severe class imbalance, complicating both model training and evaluation. A model that ”looks accurate” can still be practically useless if it misses most frauds or overwhelms analysts with false alarms.

As digital payments have expanded, fraud has increasingly shifted toward card-not-present transactions, where authorization relies primarily on card details and contextual signals. When transactions move online, possession of the physical card is no longer required, increasing exposure to fraud (Xuan et al., 2018). Credit card fraud results in substantial financial losses for banks, merchants, and consumers and undermines trust in digital payment systems. Industry reporting indicates that global payment card fraud losses reached $33.83 billion in 2023 and are projected to continue rising, with most losses occurring in online transactions (Nilson Report, 2023).

From a practical perspective, fraud detection involves significant operational trade-offs. False negatives lead to chargebacks and downstream investigations, while false positives create customer friction and lost sales (J.P.Morgan, 2025). Large volumes of reported credit card misuse further underscore that effective fraud detection requires balancing competing error costs under tight operational constraints (Federal Trade Commission, 2024). Fraud prevention also extends beyond predictive modeling into a broader ecosystem of controls. Traditional measures such as KYC procedures and rule-based screening remain necessary but are often insufficient in modern digital payment environments, where fraudsters can exploit rigid rules and delays (ACI Worldwide, 2025).

Credit card fraud detection is methodologically difficult because the data generating process exhibits two challenging properties: extreme class imbalance and strategic behavior. Kumar et al. (2022) describe class imbalance as a major obstacle, noting that naive classifiers can achieve deceptively high accuracy by predicting the majority class almost everywhere. Small changes in thresholding can dramatically alter false alert rates, and metrics like accuracy can obscure whether a model is truly useful. Beyond imbalance, fraud involves malicious intent: Xuan et al. (2018) discuss attack vectors such as phishing and malware-driven theft, and Afriyie et al. (2023) emphasize that fraudsters adapt their methods as online transactions become more common.

This study adopts a supervised learning framework, training models on historical transactions labeled as fraudulent or legitimate. While unsupervised approaches are often proposed, they rely on deviations from assumed ”normal” behavior and are more difficult to evaluate. Because our dataset provides transaction-level fraud labels, a supervised setting allows direct comparison of standard classifiers and systematic assessment of imbalance-handling techniques.

A key methodological question becomes: how should we handle imbalance? Afriyie et al. (2023) note that the dataset is ”significantly skewed” and choose undersampling, while pointing out that other work uses SMOTE (Synthetic Minority Oversampling Technique). This choice matters: undersampling discards majority-class information, whereas SMOTE creates synthetic minority examples and can change the decision boundary learned by the classifier.

1.2 Research Question

This research examines the effectiveness of supervised machine learning classifiers in detecting credit card fraud under severe class imbalance. We evaluate Random Forest and Support Vector Machine models trained on original imbalanced data and on data balanced using SMOTE. The primary research question is:

Does applying SMOTE to the training data improve fraud-detection performance for Random Forest and SVM compared to training on the original imbalanced data?

2 Methods

Figure 2.1 provides an overview of the modelling pipeline adopted in this study. The workflow is designed to ensure a clear separation between model development and final evaluation, thereby preventing information leakage and overly optimistic performance estimates. Following exploratory data analysis, feature engineering and preprocessing, the development dataset (`fraudTrain.csv`) is split into training and validation subsets. Model fitting, evaluation and hyperparameter tuning are performed on the training and validation data within a dedicated model development loop. The tuned Random Forest and linear Support Vector Machine models are then assessed on the test set under both baseline and SMOTE-enhanced configurations. This final evaluation provides an unbiased estimate of out-of-sample fraud detection performance and enables a consistent comparison across modelling approaches.

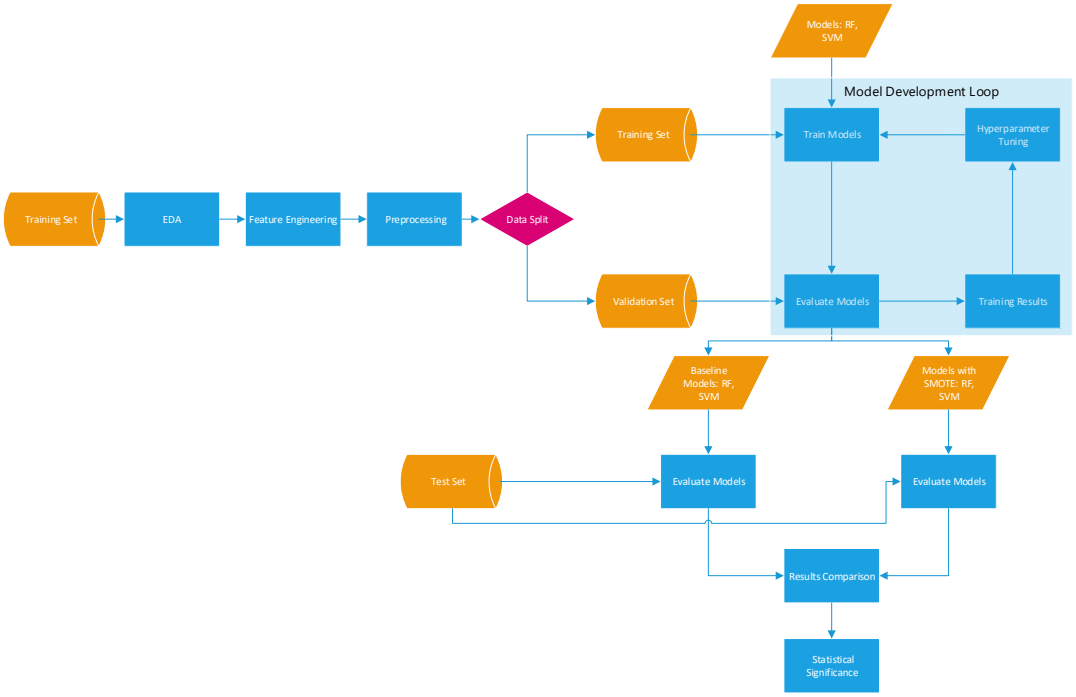

Figure 2.1: ML Pipeline

2.1 Dataset Description

The analysis is conducted on a publicly available credit card transaction dataset obtained from Kaggle (Kaggle, 2020). The dataset is provided as two separate files, `fraudTrain.csv` and `fraudTest.csv`, which are temporally disjoint and share an identical feature schema. Together, the two files correspond to an approximate 70/30 split between development and test data.

The file `fraudTrain.csv` contains 1,296,675 observations and is used as the development dataset. This dataset is further split into training and validation subsets using an 80/20 stratified split, as described in Section 2.4, and is used for model fitting, preprocessing, and hyperparameter tuning. The file `fraudTest.csv` contains 555,719 observations and serves as a held-out test set. It is not used during model development and is reserved exclusively for final out-of-sample evaluation.

Training Set

The training dataset contains approximately 1.3 million individual credit card transactions recorded between early 2019 and mid-2020. Each observation corresponds to a single transaction and is associated with a binary target variable, `is_fraud`, indicating whether the transaction was fraudulent. The dataset exhibits a pronounced class imbalance, with fraudulent transactions accounting for a small fraction of all observations.

The training data includes a heterogeneous set of features capturing transactional, temporal, geographical, and customer-related information. Transaction-level attributes include the transaction amount (`amt`), merchant identifier (`merchant`), merchant category (`category`), and transaction timestamps (`trans_date_trans_time`, `unix_time`). The dataset comprises approximately 693 unique merchants, with transaction activity concentrated in a small number of dominant merchant categories such as gas transport and grocery point-of-sale transactions.

Customer-related variables include demographic and locational information such as gender, city, state, ZIP code, city population, job title, and date of birth. Geographic coordinates are provided for both the cardholder (`lat`, `long`) and the merchant location (`merch_lat`, `merch_long`), enabling spatial feature construction. In addition, several identifier-like variables are present, including credit card numbers (`cc_num`), transaction identifiers (`trans_num`), customer names, and street-level addresses. Fraudulent transactions account for approximately 99% of all observations, underscoring the extreme imbalance characteristic of real-world fraud detection tasks.

Test Set

The held-out test dataset contains approximately 550,000 transactions recorded from mid-2020 to early 2021. It follows the same feature structure as the training data, including the same set of transactional, categorical, temporal, and geographical variables. The distribution of key attributes, such as merchant categories, transaction amounts, and demographic characteristics, is broadly consistent with the training set, while reflecting natural temporal variation. The test set is used exclusively for final model evaluation and is not involved in feature engineering, encoding, or model training.

Feature Overview.

An overview of the variables available in both datasets is provided below:

Table 2.1: Overview of features in the credit card fraud dataset.

Feature	**Description**
`index`	Unique identifier for each transaction
`trans_date_trans_time`	Transaction date and time
`cc_num`	Credit card number of the customer
`merchant`	Merchant name
`category`	Merchant category
`amt`	Transaction amount
`first`, `last`	Cardholder first and last name
`gender`	Cardholder gender
`street`, `city`, `state`, `zip`	Cardholder address information
`lat`, `long`	Cardholder geographic coordinates
`city_pop`	Population of the cardholder's city
`job`	Cardholder occupation
`dob`	Cardholder date of birth
`trans_num`	Unique transaction identifier
`unix_time`	Unix timestamp of the transaction
`merch_lat`, `merch_long`	Merchant geographic coordinates
`is_fraud`	Fraud indicator (target variable)

The presence of high-cardinality identifiers and sensitive personal attributes motivates careful feature selection and leakage prevention strategies, which are discussed in subsequent sections.

2.2 Exploratory Data Analysis

This section describes the exploratory analysis conducted to understand the structure of the data, class imbalance, and key characteristics of transactional, temporal, and categorical variables.

2.2.1 Class Imbalance

The dataset exhibits extreme class imbalance, with fraudulent transactions representing well below one percent of all observations. This imbalance highlights the inadequacy of accuracy-based evaluation and motivates the use of probability-based metrics and stratified sampling throughout model development. Figure 2.2 illustrates the extreme rarity of fraudulent transactions, which complicates reliable fraud detection and performance assessment.

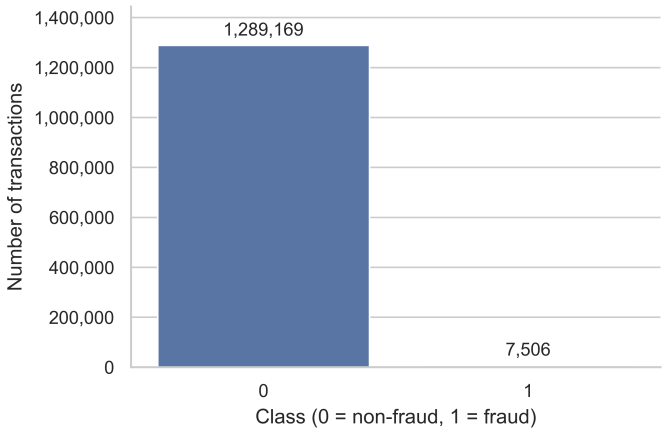

Figure 2.2: Distribution of fraudulent and non-fraudulent transactions

2.2.2 Temporal Patterns

Fraud occurrence exhibits clear temporal patterns, as shown in figure 2.3. The fraud rate varies significantly across hours of the day, with elevated rates observed during late-night hours and early morning periods. In contrast, daytime hours show substantially lower fraud incidence. Similarly, fraud rates differ across weekdays, indicating non-uniform temporal behavior over the week. These findings motivate the construction of cyclical temporal features to capture periodic patterns.

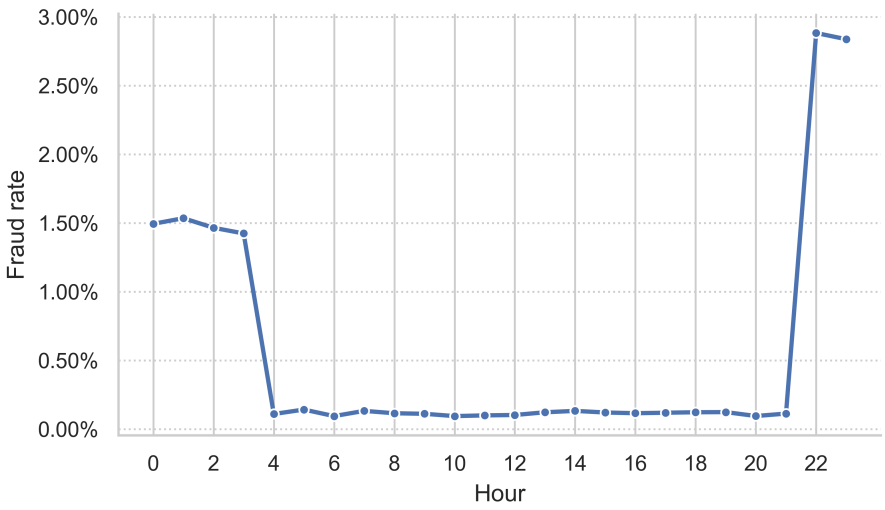

Figure 2.3: Hourly fraud rate

2.2.3 Transaction Amount

As shown in figure 2.4, transaction amounts are highly right-skewed and span several orders of magnitude. Fraudulent transactions tend to involve higher amounts and greater dispersion than legitimate ones, although substantial overlap between classes remains, indicating that transaction amount alone is insufficient for reliable fraud detection.

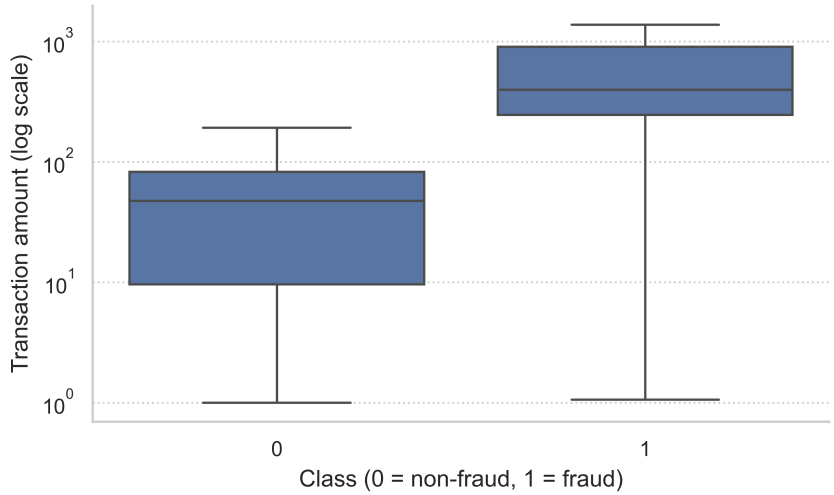

Figure 2.4: Distribution of transaction amounts by class on a logarithmic scale

2.2.4 City Population

City population distributions are highly skewed, spanning several orders of magnitude. Fraudulent transactions tend to occur slightly more frequently in locations with larger populations, although the overlap between fraudulent and non-fraudulent distributions remains substantial. City population is therefore treated as a weak but potentially informative contextual feature.

2.2.5 Feature Correlations

Correlation analysis among numeric and engineered temporal features reveals limited linear dependence between most variables and the fraud label. Strong correlations are primarily observed among derived temporal features (e.g., sine and cosine representations), reflecting their mathematical construction. The absence of strong linear correlations with the target suggests that nonlinear models may be better suited to capture complex fraud patterns.

2.2.6 High-Cardinality Features

Several categorical variables, including merchant, city, job, and ZIP code, exhibit high cardinality and long-tailed frequency distributions. As shown in Figure 2.5, fraud rates vary substantially across merchant categories, indicating heterogeneous category-level risk. These properties motivate the use of encoding strategies specifically designed for high-cardinality categorical features.

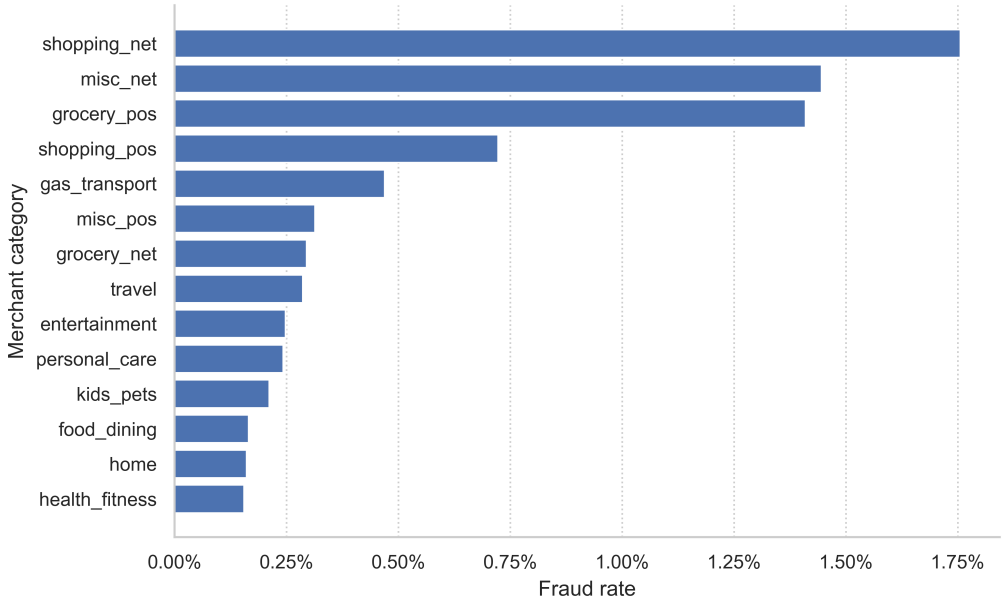

Figure 2.5: Fraud rates across top merchant categories

2.3 Feature Engineering

This section outlines the feature engineering procedures applied prior to preprocessing, focusing on deterministic and domain-driven transformations.

2.3.1 Temporal Feature Construction

Transaction timestamps are transformed into a set of interpretable and model-friendly temporal features. First, the raw transaction datetime is converted into calendar-based components, including the hour of the day, day of the week, and month. In addition, a binary weekend indicator is constructed to distinguish between weekday and weekend transactions.

To appropriately capture the cyclical nature of temporal variables, sine and cosine transformations are applied to the hour-of-day, weekday, and month components. This encoding preserves the periodic structure of time (e.g., the proximity between 23:00 and 00:00) and avoids artificial discontinuities that arise from linear representations. Formally, for a cyclical variable t with period T, the transformation is given by

$$\sin\left(\frac{2\pi t}{T}\right), \quad \cos\left(\frac{2\pi t}{T}\right).$$

After constructing these cyclical representations, the original raw timestamp and intermediate calendar variables are removed to prevent redundancy and potential leakage of ordering information.

In addition, customer age is derived from the date of birth by computing the difference between the transaction date and the recorded birth date, expressed in years. The raw date-of-birth field is subsequently removed, leaving age as a continuous demographic feature.

2.3.2 Categorical Feature Encoding

Low-cardinality categorical variables are encoded using one-hot encoding, while high-cardinality variables are encoded using target encoding learned exclusively from the training data.

2.3.3 Feature Exclusion and Leakage Prevention

Several identifier-like variables, including credit card numbers, transaction identifiers, cardholder names, street-level addresses, and raw timestamps, are deliberately excluded from the feature set. Exploratory analysis reveals that these variables exhibit extreme entity-level fraud concentration, with multiple entities displaying fraud rates of 1.0 supported by repeated observations. While such patterns reflect genuine historical fraud concentration, they enable memorization of individual entities rather than learning generalizable fraud patterns.

This phenomenon is a well-documented form of target leakage and entity memorization in supervised learning. Kaufman et al. (2012) demonstrate that high-cardinality identifiers often exhibit strong apparent predictive power due to data collection artifacts and sampling bias, leading to overly optimistic performance estimates. In the context of fraud detection, Dal Pozzolo et al. (2015) emphasize that repeated entity occurrences can bias evaluation and hinder generalization under temporal and behavioral drift.

In line with these findings, all identifier-based variables are excluded to prevent leakage and ensure that the models learn transferable behavioral patterns rather than entity-specific histories.

2.4 Data Splitting Strategy

To ensure a robust evaluation, we follow a three way splitting strategy that mirrors the workflow implemented in our Python pipeline. The dataset is provided in two separate files, `fraudTrain.csv` and `fraudTest.csv`, which are temporally disjoint. We interpret `fraudTrain.csv` as the historical development sample used for model building and hyperparameter tuning, while `fraudTest.csv` is treated as a final held out evaluation sample that is never used during model selection.

2.4.1 Development Sample: Training and Validation Split

All preprocessing and model selection steps are performed exclusively on `fraudTrain.csv`. From this development dataset, we create an internal validation set via a stratified random split:

$$(X_{\text{train}}, y_{\text{train}}),\ (X_{\text{val}}, y_{\text{val}}) \leftarrow \texttt{train\_test\_split}(X, y, \texttt{test\_size} = 0.2, \texttt{stratify} = y, \texttt{random\_state} = 42).$$

Stratification preserves the severe class imbalance in both subsets, ensuring that the fraud prevalence in the validation sample remains representative of the development data. The training subset (80%) is used to fit model parameters, including all preprocessing transformations, while the validation subset (20%) is used for hyperparameter selection and model comparison. This results in an 80/20 split between training and validation data within the original Kaggle training set.

2.4.2 External Test Sample

The file `fraudTest.csv` is used as an external test set $(X_{\text{test}}, y_{\text{test}})$. This split is defined by the dataset provider and is temporally separated from the training data, which supports a realistic out of sample assessment under potential temporal variation. Importantly, `fraudTest.csv` is not used for hyperparameter tuning, feature selection, calibration decisions, or any other model development step. It is only used once at the end to report final model performance.

2.4.3 Preprocessing within the Split and Leakage Prevention

All feature engineering steps, including the construction of cyclical time features (sine and cosine transformations) and the derivation of age from date of birth, are applied consistently to the training, validation, and test sets. However, to avoid information leakage, model fitting related transformations are learned only on the training portion of the development sample. Concretely, the preprocessing pipeline includes:

- Standardization of numerical features using `StandardScaler`,
- One hot encoding for low cardinality categorical variables using `OneHotEncoder`,
- Target encoding for high cardinality categorical variables using `TargetEncoder`.

These transformations are implemented within a single pipeline object, such that `fit` is applied only on $(X_{\text{train}}, y_{\text{train}})$ and the resulting transformations are subsequently transformed and applied to X_{val} and X_{test}. This ensures that scaling parameters, one hot encoding categories, and target encoding mappings are estimated solely from the training subset and do not incorporate information from validation or test observations.

2.4.4 Model Selection versus Final Evaluation

The internal validation set is used for model selection. In particular, for the Support Vector Machine model we tune the regularization parameter C on the validation data, while for the Random Forest model we explore key structural hyperparameters such as tree depth, minimum leaf size, and the number of features considered at each split. In both cases, the selected configuration maximizes the chosen validation metric. For both models, hyperparameter selection is based on probability-based validation metrics, with particular emphasis on precision–recall performance due to extreme class imbalance. After selecting the best configuration, we report two sets of results:

- **Internal validation performance** on $(X_{\text{val}}, y_{\text{val}})$, which reflects model selection quality within the development sample,
- **External test performance** on $(X_{\text{test}}, y_{\text{test}})$, which reflects final generalization to unseen and temporally separated data.

This separation between development and final evaluation is critical in fraud detection settings, where class imbalance and potential temporal drift can otherwise lead to overly optimistic performance estimates if the test set is used repeatedly during tuning.

2.5 Handling Class Imbalance

Class imbalance poses a significant challenge for supervised learning algorithms, as standard classifiers tend to be biased toward the majority class. To address this issue, we apply the Synthetic Minority Over-sampling Technique (SMOTE), originally proposed by Chawla et al. (2002), which generates artificial minority class samples via interpolation in feature space.

For each minority class observation $\mathbf{x}_i \in \mathbb{R}^d$, k nearest neighbors are identified within the minority class using Euclidean distance. Synthetic samples are generated by selecting a neighbor $\mathbf{x}_{\text{nn}}$ at random and computing

$$\mathbf{x}_{\text{synthetic}} = \mathbf{x}_i + \lambda \left(\mathbf{x}_{\text{nn}} - \mathbf{x}_i \right), \tag{2.1}$$

where $\lambda \sim \mathcal{U}(0, 1)$. This procedure places new samples along line segments between neighboring minority observations, promoting more general decision regions.

In this study, SMOTE is implemented using the `scikit-learn`-compatible implementation from the `imbalanced-learn` library, which closely follows the original formulation of Chawla et al. (2002), with $k = 5$ by default. All synthetic samples are generated using training data only to avoid information leakage. Models trained with SMOTE are evaluated on the held out test set, which preserves the original class distribution. This design ensures a clear separation between model development (training and internal validation on `fraudTrain.csv`) and final evaluation (testing on `fraudTest.csv`).

2.6 Random Forest Model

2.6.1 Model Choice and Motivation

Random Forest is employed as a nonlinear ensemble classifier capable of capturing complex interactions among transactional, temporal, and categorical features. The method constructs a collection of decision trees trained on bootstrapped samples of the training data and aggregates their predictions through majority voting or probability averaging. Compared to single decision trees, Random Forest reduces variance and improves generalisation by decorrelating individual trees through random feature selection at each split.

Random Forest has been widely adopted in credit card fraud detection due to its robustness to noise, ability to model heterogeneous feature types, and strong empirical performance

under severe class imbalance. Prior work by Xuan et al. (2018) demonstrates that Random Forest models outperform several classical classifiers in fraud detection tasks and remain effective even when fraudulent observations are extremely rare. Breiman (2001) formalises Random Forest as an ensemble method that balances individual tree strength and inter tree diversity, yielding stable predictive performance in high dimensional settings.

2.6.2 Model Specification

Random Forest extends the bagging principle by training each decision tree on a bootstrap sample of the training data and further decorrelating trees through random feature selection at each split. At each internal node, a random subset of features is considered when selecting the split criterion, following the standard square root heuristic for classification problems. Trees are grown to full depth unless constrained by explicit stopping parameters.

In the implementation, Random Forest is embedded within the same preprocessing pipeline used for all models. Numerical features are standardised, low cardinality categorical variables are one hot encoded, and high cardinality categorical variables are target encoded. All preprocessing steps are fitted exclusively on the training subset to prevent information leakage.

In a separate model variant, SMOTE is applied to the training subset only (Section 2.5). Oversampling is implemented within the pipeline such that synthetic minority samples are generated exclusively during model fitting and never on validation or test data, thereby preventing evaluation leakage.

2.6.3 Hyperparameter Tuning

A structured hyperparameter tuning procedure is conducted on the internal validation set to examine the sensitivity of Random Forest performance to key hyperparameters. The explored parameters include the number of trees, maximum tree depth, minimum number of samples required at leaf nodes, and the number of features considered at each split. These parameters directly control the bias variance trade off and the degree of model regularisation.

Rather than evaluating all possible combinations of the specified hyperparameter values, the tuning procedure focuses on a targeted set of configurations centred around a baseline model summarised as

$$
\begin{aligned}
n_{\text{estimators}} &\in \{300, 500\}, \\
\text{max\_depth} &\in \{\text{None}, 10, 20\}, \\
\text{min\_samples\_leaf} &\in \{1, 5, 10\}, \\
\text{max\_features} &\in \{\text{sqrt}, 0.3, 0.5\}.
\end{aligned}
$$

To control computational cost, only a subset of combinations is evaluated, primarily varying one hyperparameter at a time around the baseline configuration.

The evaluated configurations reveal that shallow trees and large minimum leaf sizes lead to underfitting, while very small leaf sizes increase variance and false positive rates. Models with unrestricted depth and small leaf sizes yield stronger ranking performance as measured by probability based metrics. Variation in the number of trees beyond a moderate ensemble size provides diminishing returns, consistent with theoretical and empirical findings in Breiman (2001).

2.7 Support Vector Machine

2.7.1 Model Choice and Objective

A linear Support Vector Machine (SVM) classifier is employed to detect fraudulent transactions in a high-dimensional feature space that includes standardized numerical variables, one-hot encoded low-cardinality categorical variables, and target-encoded high-cardinality categorical variables. A linear SVM is well suited to this setting because it scales efficiently to large datasets and constructs a single separating hyperplane in the transformed feature space.

The objective of the linear SVM is to find a decision boundary that maximizes the margin between fraudulent and non-fraudulent transactions while allowing for a limited number of classification errors. Misclassified observations are penalized through a regularization mechanism that balances margin width against training errors. This trade-off is controlled by a regularization parameter, which determines how strongly the model prioritizes correct classification of training samples versus generalization to unseen data.

By optimizing this margin-based objective, the linear SVM produces a robust classifier that is less sensitive to noise and high-dimensional feature representations, making it suitable for large-scale fraud detection tasks.

In a separate model variant, SMOTE is additionally applied to the training subset only (Section 2.5). SMOTE is implemented within the pipeline such that synthetic minority

samples are generated exclusively during model fitting and never on validation or test data, thereby preventing evaluation leakage.

2.7.2 Probability Calibration

A standard linear SVM produces decision scores but not calibrated probabilities. Since evaluation and threshold selection require probability like outputs, the SVM is wrapped in `CalibratedClassifierCV`, which fits a calibration model on top of the SVM decision function and yields $\hat{p}(y = 1 \mid \mathbf{x})$. Calibration is performed using cross validation on the training data, ensuring that the calibration mapping is learned without using held out evaluation samples.

2.7.3 Hyperparameter Selection for C

The regularization parameter C is evaluated over a log spaced grid

$$C \in \{0.01, 0.1, 1, 10, 100, 1000\}.$$

Each candidate model is trained on the training portion of the development sample and evaluated on the internal validation set. ROC–AUC and PR–AUC are reported based on the predicted probabilities. Empirical results show only marginal variation across the tested values of C, indicating that further tuning of C provides limited benefit in this setting. Consequently, the default value $C = 1$ is used for the main threshold tuning analysis.

2.8 Decision Threshold Selection

Although the Random Forest classifier outputs calibrated class probabilities, operational fraud detection requires a binary decision rule. Under extreme class imbalance, a fixed default threshold is suboptimal, as it prioritises majority-class accuracy at the expense of fraud recall. While ROC–AUC and PR–AUC assess the ranking quality of predicted scores, they do not determine an actionable classification decision. Therefore, the classification threshold τ is explicitly tuned on the validation set using predicted fraud probabilities $\hat{p}(y = 1 \mid \mathbf{x})$.

A default threshold of $\tau = 0.5$ is reported as a baseline. Operational thresholds are evaluated along the precision–recall curve, and the selected threshold $\tau^\star$ is obtained by

maximising the F_2 score:

$$\tau^\star = \arg\max_\tau F_2(\tau), \qquad F_2(\tau) = \frac{(1+2^2)\,\text{Precision}(\tau)\,\text{Recall}(\tau)}{2^2\,\text{Precision}(\tau) + \text{Recall}(\tau)}.$$

This criterion places greater emphasis on recall, reflecting the higher cost of missed fraudulent transactions, while retaining sensitivity to false positives. The selected threshold is subsequently applied unchanged to the external test set to obtain an unbiased estimate of the operational trade-off between fraud detection performance and false alarm volume.

2.9 Evaluation on Test Set

The hyperparameter tuned Random Forest and Support Vector Machine models serve as baseline classifiers for comparison with imbalance aware variants, including models trained using SMOTE. Model performance is evaluated using both threshold dependent and probability based metrics. Class specific precision, recall, and F_1 scores are reported to quantify fraud detection capability and false alarm volume under the selected operational decision rule. In addition, ranking performance is assessed using the area under the receiver operating characteristic curve (ROC–AUC) and the area under the precision recall curve (PR–AUC). Under extreme class imbalance, PR–AUC is particularly informative, as it focuses on minority class performance and is sensitive to changes in false positive rates.

Precision recall curves are visualised to illustrate how classification performance varies across decision thresholds. The default classifier threshold and the F_2 optimal threshold are highlighted to demonstrate the impact of explicit threshold tuning on the trade off between fraud recall and precision. Together, these results provide an unbiased estimate of out of sample performance and establish a consistent baseline against which imbalance handling techniques are evaluated.

2.10 Statistical Significance

To assess whether the observed performance differences between models trained with and without SMOTE are statistically significant, we adopt a non-parametric testing approach tailored to binary classification on a single held-out test set (Rainio et al., 2024). Due to the presence of temporal dependencies in the transaction data, resampling-based evaluation procedures such as standard cross-validation are deliberately avoided, as they would violate the chronological structure of the data and potentially introduce look-ahead bias. Instead,

model evaluation is performed on a temporally held-out test set that is not used during model training or hyperparameter tuning.

Statistical significance is assessed using McNemar's test, which is specifically designed for comparing two classifiers evaluated on the same test instances using paired predictions. The test examines whether the two models differ significantly in their error patterns, focusing on the instances for which the models disagree. In line with the fraud detection setting, McNemar's test is applied exclusively to observations belonging to the fraud (positive) class, thereby testing for a statistically significant difference in recall (sensitivity) between models trained with and without SMOTE.

For each model pair (SVM vs. SVM+SMOTE and Random Forest vs. Random Forest+SMOTE), predictions are generated on the same held-out test set, ensuring a paired comparison at the instance level. The null hypothesis states that both models commit the same number of classification errors on fraud cases, while rejection of the null hypothesis indicates a statistically significant difference in fraud detection capability.

Statistical significance is evaluated at the 5% level. Alongside hypothesis testing, fraud-class precision and recall computed on the held-out test set are reported to provide an interpretable assessment of model performance. Precision is reported descriptively, as it is not directly amenable to McNemar's test due to its dependence on predicted positive counts rather than paired instance-level outcomes.

2.11 Usage of Artificial Intelligence

For this paper some artificial intelligence (AI) was used. The tools included are OpenAI's ChatGPT (Version 5), which provided support with programming tasks and the generation of text suggestions, and DeepL's DeepL Translator which was employed for the translation of individual text passages. Both tools contributed to the efficiency and accuracy of the project.

3 Results

3.1 Training

3.1.1 Random Forest Hyperparameter Tuning

Table 3.1 summarises validation ROC-AUC and PR-AUC for a targeted set of Random Forest hyperparameter configurations evaluated on the internal validation set. Ranking performance is consistently strong across all configurations, with ROC-AUC values above 0.988. In contrast, PR-AUC exhibits greater variation, highlighting meaningful differences in the models' ability to rank fraudulent transactions.

Models with unrestricted tree depth perform best overall. Imposing depth constraints leads to a noticeable degradation in PR-AUC, with the shallowest configuration (max_depth = 10) performing worst. Among unrestricted-depth models, the combination min_samples_leaf = 10 and max_features = 0.5 achieves the highest PR-AUC (0.926). Increasing the number of trees from 300 to 500 yields negligible improvements, indicating diminishing returns beyond a moderate ensemble size. Configurations with very small leaf sizes (min_samples_leaf = 1) remain competitive in terms of PR-AUC (0.923) but are less regularised and therefore more prone to variance.

Based on these results, the configuration max_depth = None, min_samples_leaf = 10, max_features = 0.5, and $n_{\text{estimators}} = 300$ is selected for subsequent threshold tuning and evaluation on the external test set.

Table 3.1: Validation-set ranking performance for selected Random Forest hyperparameter configurations.

Configuration	**ROC-AUC**	**PR-AUC**
max_depth = None, min_leaf = 10, max_feat = 0.5	0.999	0.926
max_depth = None, min_leaf = 1, max_feat = sqrt	0.996	0.923
max_depth = None, min_leaf = 10, max_feat = 0.3	0.998	0.920
max_depth = None, min_leaf = 5, max_feat = sqrt	0.997	0.904
max_depth = 20, min_leaf = 10, max_feat = sqrt	0.995	0.879
max_depth = 10, min_leaf = 10, max_feat = sqrt	0.988	0.798

3.1.2 Support Vector Machine: Hyperparameter Selection

Table 3.2 reports validation ROC-AUC and PR-AUC across the regularization parameter C for the linear SVM classifier with balanced class weights. Performance is relatively stable across all tested values of C, with ROC-AUC remaining above 0.95 and PR-AUC ranging between 0.366 and 0.377. This stability indicates that ranking performance is largely insensitive to the precise regularization strength under extreme class imbalance.

The results indicate minimal sensitivity to the choice of C in this setting. Given the negligible differences in ranking performance, the default value $C = 1$ is retained for subsequent analysis.

Table 3.2: Validation-set ranking performance of the SVM model under different regularization parameter values.

C	**ROC-AUC**	**PR-AUC**
0.01	0.954	0.366
0.10	0.955	0.376
1.00	0.955	0.377
10.00	0.955	0.376
100.00	0.955	0.377
1000.00	0.955	0.377

3.1.3 Decision Threshold Selection (Validation Set)

Operational deployment requires a binary decision rule. For each model, we report performance at the default threshold $\tau = 0.5$ and at the threshold that maximises the F_2 score on the internal validation set. Table 3.3 summarises the resulting operating points and the corresponding fraud-class precision, recall, and F_2 score.

Table 3.3: Validation-set fraud-class performance under default and F_2-optimal decision thresholds.

Model	**Threshold**	**Fraud Precision**	**Fraud Recall**	**F_2**
SVM	0.50 (default)	0.52	0.10	0.12
SVM	0.025 (F_2-opt)	0.34	0.63	0.54
RF	0.50 (default)	0.96	0.79	0.86
RF	0.201 (F_2-opt)	0.83	0.88	0.85

For the Random Forest model, the default threshold already yields strong fraud-class performance, with high precision and recall. Optimising the threshold for the F_2 score

results in a modest increase in recall accompanied by a moderate reduction in precision, reflecting a favourable precision–recall trade-off.

In contrast, the Support Vector Machine exhibits a strong bias toward the majority class at the default threshold, achieving high precision but very low recall. F_2-based threshold tuning substantially shifts the operating point toward higher recall at the cost of reduced precision, yielding a marked improvement in recall-weighted performance.

Figure 3.1 and Figure 3.2 show the precision–recall curves for the Random Forest and Support Vector Machine models on the validation set. The red point indicates the default threshold ($\tau = 0.5$), and the green point indicates the F_2-optimal threshold.

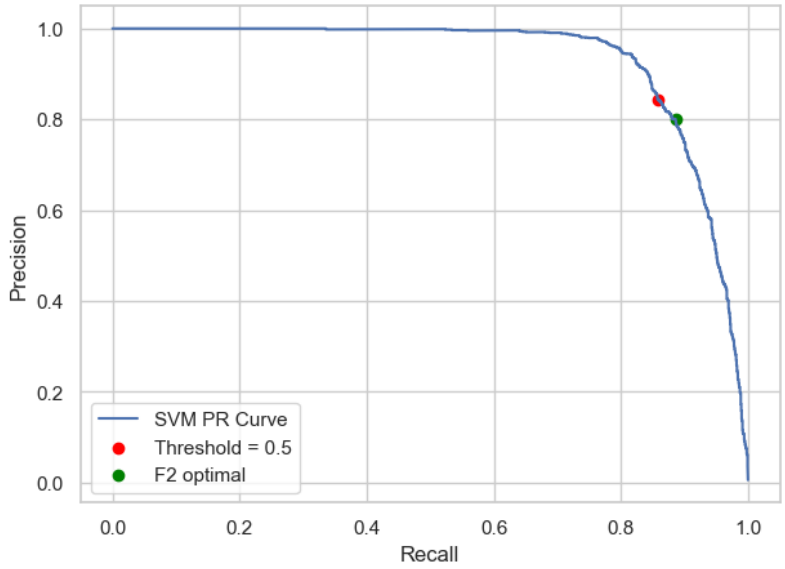

Figure 3.1: Precision–recall curve for the Random Forest model on the validation set

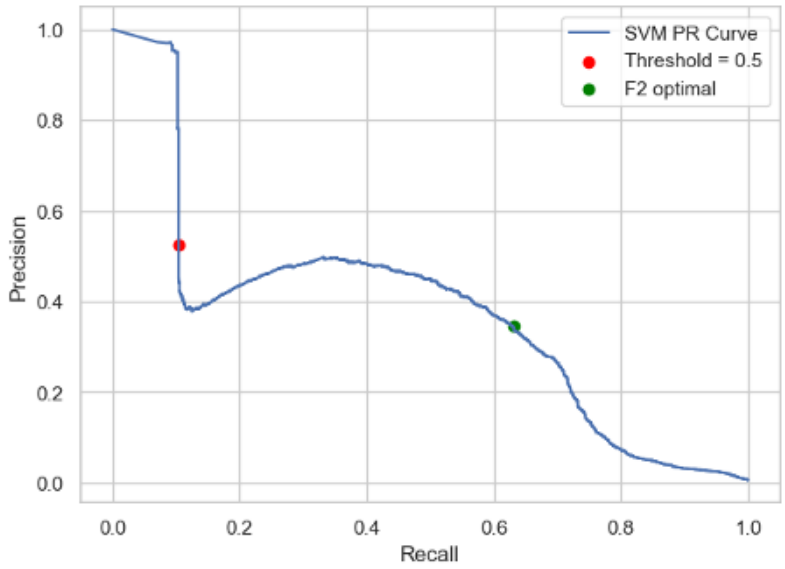

Figure 3.2: Precision–recall curve for the Support Vector Machine model

3.2 Model Comparison on the Test Set

Table 3.4 reports the test-set classification performance of the hyperparameter-tuned Random Forest and SVM models, evaluated using an F_2-optimised decision threshold. Both models achieve near-perfect classification of non-fraudulent transactions, reflecting the pronounced class imbalance in the dataset.

For the fraud class, the Random Forest attains a precision of 0.59 and a recall of 0.41, corresponding to an F_1-score of 0.48. The SVM achieves a similar recall of 0.38 but substantially lower precision of 0.18, resulting in an F_1-score of 0.24. Overall, while both models identify a comparable fraction of fraudulent transactions, the Random Forest produces considerably fewer false positives and therefore yields superior minority-class performance.

Table 3.4: Test-set classification performance of the Random Forest and SVM models.

Model	**Class**	**Precision**	**Recall**	**F1-score**	**Support**
Random Forest	Non-fraud (0)	1.00	1.00	1.00	553 574
	Fraud (1)	0.59	0.41	0.48	2 145
SVM	Non-fraud (0)	1.00	1.00	1.00	553 574
	Fraud (1)	0.18	0.38	0.24	2 145

3.2.1 SMOTE

Table 3.5 reports the test-set performance of the Random Forest and SVM models trained with SMOTE-based oversampling. Despite oversampling the minority class during training, both models exhibit degraded fraud detection performance compared to their non-resampled counterparts.

For the Random Forest, the fraud class achieves a precision of 0.10 and a recall of 0.19, corresponding to an F_1-score of 0.13. This indicates that only a small fraction of fraudulent transactions are correctly identified, while a large proportion of flagged transactions are false positives.

The SVM trained with SMOTE exhibits an even more extreme trade-off, achieving a fraud recall of 0.98 but a precision of only 0.01, resulting in an F_1-score of 0.01. This behavior reflects an overly aggressive decision boundary that labels nearly all transactions as fraudulent, leading to an impractically high false-positive rate.

As in previous experiments, performance on the non-fraud class remains high due to the pronounced class imbalance. Overall, these results indicate that SMOTE-based oversampling does not improve and instead substantially degrades the generalization performance of both Random Forest and SVM classifiers in this setting.

Table 3.5: Test-set classification performance of Random Forest and SVM models trained with SMOTE-based oversampling

Model	Class	Precision	Recall	F1-score	Support
Random Forest (SMOTE)	Non-fraud (0)	1.00	0.99	1.00	553 574
	Fraud (1)	0.10	0.19	0.13	2 145
SVM (SMOTE)	Non-fraud (0)	1.00	0.37	0.54	553 574
	Fraud (1)	0.01	0.98	0.01	2 145

3.2.2 Statistical Significance on Fraud Recall

To assess whether to assess whether SMOTE-based oversampling leads to statistically significant changes in fraud recall, we apply McNemar's test on held-out test set. The test is restricted to true fraud cases and evaluates differences in recall between models trained with and without SMOTE-based oversampling. Since both models are evaluated on the same observations, McNemar's test provides a paired comparison based on discordant predictions.

Table 3.6 summarizes the results for both Random Forest and SVM classifiers. For Random Forest, the model trained without SMOTE correctly identifies substantially more

fraud cases than its SMOTE-based counterpart, resulting in a decisive rejection of the null hypothesis. In contrast, SMOTE significantly increases fraud recall for the SVM. However, as shown in the previous subsection, this increase in recall is accompanied by a severe deterioration in precision. Here, b denotes fraud cases missed by the non-SMOTE model but detected by the SMOTE-based model, while c denotes fraud cases detected by the non-SMOTE model but missed by the SMOTE-based model.

Table 3.6: McNemar test results comparing fraud recall between models trained with and without SMOTE-based oversampling.

Model	b	c	$b + c$	**Test statistic**	**p-value**	**Effect on recall**
Random Forest	12	453	465	489.99	$< 10^{-100}$	SMOTE worse
SVM	647	0	647	640.00	$< 10^{-100}$	SMOTE better

4 Discussion & Conclusion

4.1 Discussion

This study investigated whether applying SMOTE to the training data improves fraud-detection performance for Random Forest and SVM compared to training on the original imbalanced data. The empirical results provide a clear and consistent answer: SMOTE-based oversampling does not improve fraud-detection performance in this setting and, in fact, substantially degrades generalization on the held-out test set for both classifiers.

4.1.1 Interpretation of Results and Comparison with Prior Work

The Random Forest trained on imbalanced data achieved fraud-class precision of 0.59 and recall of 0.41 (F_1=0.48) on the external test set. With SMOTE, precision dropped to 0.10 and recall to 0.19 (F_1=0.13), representing substantial deterioration across all metrics. McNemar's test confirmed this difference as statistically significant, with the non-SMOTE model identifying 453 more fraud cases.

For the SVM, the pattern differs but is equally problematic. The SMOTE-trained SVM achieved near-perfect recall (0.98) but impractically low precision (0.01), reflecting an overly aggressive decision boundary that classifies nearly all transactions as fraudulent, rendering it operationally useless. The baseline SVM achieved more balanced performance with precision of 0.18 and recall of 0.38.

These findings suggest that SMOTE's synthetic samples introduce distributional artifacts that do not generalize to genuine out-of-sample fraud patterns. The interpolation-based generation may produce feature combinations plausible in training but unrealistic in the temporally separated test set.

This negative effect contrasts with prior findings: Chawla et al. (2002) demonstrated benefits of synthetic oversampling, and Afriyie et al. (2023) reported improved performance using undersampling. However, these studies differ methodologically, often using cross-validation rather than temporally disjoint test sets. Our results align with Dal Pozzolo et al. (2015), who emphasize that temporal dependencies and behavioral drift require careful evaluation, and that techniques effective in controlled settings may fail under realistic deployment conditions.

4.1.2 Role of Threshold Selection

An important finding of this study is that explicit threshold tuning on the validation set substantially improves operational performance, particularly for the SVM classifier. At the default threshold of 0.5, the baseline SVM achieved a fraud recall of only 0.10, which increased to 0.63 after F_2-based threshold optimization. This demonstrates that, under extreme class imbalance, careful calibration of the decision threshold is essential for translating ranking performance into actionable fraud detection.

For the Random Forest model, the default threshold already yielded strong performance, with only modest gains from further optimization. This difference reflects the Random Forest's superior ranking ability and its capacity to produce well-calibrated probability estimates even without post-hoc adjustment.

4.1.3 Limitations

Several limitations should be acknowledged. First, the study relies on a single publicly available dataset, which, while realistic in structure, may not capture the full heterogeneity of fraud patterns encountered in different financial institutions or geographic regions. Second, only SMOTE was evaluated as an oversampling technique; alternative resampling strategies such as undersampling or hybrid approaches were not considered. Third, the feature engineering choices, while motivated by leakage prevention and domain considerations, may have excluded potentially informative signals.

4.2 Conclusion

This study addressed the research question: *Does applying SMOTE to the training data improve fraud-detection performance for Random Forest and SVM compared to training on the original imbalanced data?*

The answer, based on systematic evaluation on a temporally separated test set, is clearly negative. SMOTE-based oversampling substantially degraded the fraud-detection performance of both Random Forest and Support Vector Machine classifiers. For the Random Forest, all fraud-class metrics declined markedly, with statistically significant reductions in recall confirmed by McNemar's test. For the SVM, SMOTE produced an extreme and impractical trade-off characterized by near-total recall but negligible precision.

These findings have important practical implications. First, they caution against the uncritical application of resampling techniques in fraud detection, particularly when evaluation

is conducted on genuinely out-of-sample data subject to temporal drift. Second, they highlight the value of explicit threshold tuning as a complementary strategy for managing class imbalance, which can yield substantial improvements in operational performance without modifying the training distribution. Third, they underscore the importance of rigorous evaluation protocols that preserve temporal ordering and prevent information leakage.

In summary, the empirical evidence from this study does not support the use of SMOTE for improving fraud detection in the evaluated setting. Instead, training on the original imbalanced data combined with careful threshold selection and robust feature engineering yields superior generalization performance. Future research may explore alternative imbalance-handling strategies, more expressive model architectures, or adaptive approaches that account for temporal non-stationarity in fraud patterns.

Bibliography

ACI Worldwide. (2025). *Driving Up Conversion with Effective Fraud Management* (tech. rep.). ACI Worldwide. https://www.aciworldwide.com/wp-content/uploads/2021/04/using-fraud-management-to-drive-up-conversation-rates-tl-a4.pdf

Afriyie, J. K., Tawiah, K., Pels, W. A., Addai-Henne, S., Dwamena, H. A., Owiredu, E. O., Ayeh, S. A., & Eshun, J. (2023). A supervised machine learning algorithm for detecting and predicting fraud in credit card transactions. *Decision Analytics Journal*, *6*, 100163. https://doi.org/10.1016/j.dajour.2023.100163

Breiman, L. (2001). Random Forests. *Springer Nature - Machine Learning*, *45*(2), 5–32. https://doi.org/https://doi.org/10.1023/A:1010933404324

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, *16*, 321–357. https://doi.org/10.1613/jair.953

Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2015). Credit card fraud detection and concept-drift adaptation with delayed supervised information. *2015 International Joint Conference on Neural Networks (IJCNN)*, 1–8. https://doi.org/10.1109/IJCNN.2015.7280527

Federal Trade Commission. (2024). Consumer Sentinel Network Data Book 2024. https://www.ftc.gov/system/files/ftc_gov/pdf/csn-annual-data-book-2024.pdf

J.P.Morgan. (2025). J.P.Morgan. https://www.jpmorgan.com/insights/payments/data-intelligence/cnp-fraud-prevention-combat-chargebacks

Kaggle. (2020). *Fraud detection dataset* [Accessed: 2026-01-22]. https://www.kaggle.com/datasets/kartik2112/fraud-detection

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, *6*(4), 1–21. https://doi.org/10.1145/2382577.2382579

Kumar, S., Gunjan, V. K., Ansari, M. D., & Pathak, R. (2022). Credit Card Fraud Detection Using Support Vector Machine [Series Title: Lecture Notes in Networks and Systems]. In V. K. Gunjan & J. M. Zurada (Eds.), *Proceedings of the 2nd International Conference on Recent Trends in Machine Learning, IoT, Smart Cities and Applications* (pp. 27–37, Vol. 237). Springer Nature Singapore. https://doi.org/10.1007/978-981-16-6407-6_3

Nilson Report. (2023). *Card Fraud Losses Worldwide in 2023* (tech. rep. No. 1276). https://nilsonreport.com/articles/card-fraud-losses-worldwide-in-2023/

Rainio, O., Teuho, J., & Klén, R. (2024). Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, *14*(1), 6086. https://doi.org/10.1038/s41598-024-56706-x

Xuan, S., Liu, G., Li, Z., Zheng, L., Wang, S., & Jiang, C. (2018). Random forest for credit card fraud detection. *2018 IEEE 15th International Conference on Networking, Sensing and Control (ICNSC)*, 1–6. https://doi.org/10.1109/ICNSC.2018.8361343

List of Tools

OpenAI (2025/2026). ChatGPT (Version 5.2). https://chat.openai.com/chat

– Support with programming

– Generation of text suggestions

Anthropic (2025/2026). Claude AI (Claude 3.Opus). https://www.anthropic.com/claude

– Support with programming and code structuring

DeepL (2025/2026). DeepL Translator (DeepL Pro). https://www.deepl.com/translator

– Translation of individual text passages

A Appendix

A.1 Alternative Imbalance Handling via Class Weighting

In addition to the SMOTE-based approach adopted in the main analysis, we also explored cost-sensitive learning as an alternative strategy for handling class imbalance. Specifically, class imbalance is addressed by adjusting class weights in the Random Forest and Support Vector Machine objectives, thereby increasing the penalty associated with misclassifying fraudulent transactions. These experiments are reported here for completeness but are not used in the final models.

Several weighting schemes are evaluated, including uniform weighting, automatic inverse-frequency weighting, and manually specified fraud-class penalties. The explored configurations are

$$\text{class\_weight} \in \Big\{\text{None, balanced, } \{0:1, 1:10\}, \{0:1, 1:50\}, \{0:1, 1:100\}, \{0:1, 1:172\}\Big\},$$

where larger weights increase the loss contribution associated with fraudulent observations.

A.1.1 Effect of Class Weighting on Random Forest

Table A.1 summarises validation-set ranking performance of the Random Forest model under alternative class-weight configurations. Differences across configurations are small and do not materially affect ranking performance. ROC-AUC remains consistently high across all settings (≥ 0.99), while PR-AUC exhibits only marginal variation.

Moderate cost sensitivity yields a slight improvement in PR-AUC, with the configuration $\{0:1,\ 1:10\}$ achieving the highest value (0.887), compared to 0.878 for the unweighted model. However, these gains are modest and do not persist under more aggressive weighting schemes. Given the limited improvement relative to the added complexity, class weighting is not pursued further in the main analysis, and the unweighted Random Forest is retained.

Table A.1: Validation-set ranking performance of the Random Forest model under different class-weight configurations.

Class weight	ROC-AUC	PR-AUC
None	0.994803	0.878254
`balanced`	0.996525	0.892846
{0:1, 1:10}	0.996145	0.887001
{0:1, 1:50}	0.996411	0.890349
{0:1, 1:100}	0.996434	0.892859
{0:1, 1:172}	0.996716	0.891176

A.1.2 Effect of Class Weighting on Support Vector Machine

Table A.2 compares validation-set ranking performance under alternative class-weight configurations for the SVM model with $C = 1$. ROC-AUC remains stable across all settings (≥ 0.95), while PR-AUC shows modest variation.

Moderate cost sensitivity with {0:1, 1:50} achieves the highest PR-AUC (0.402), outperforming the automatic `balanced` weighting scheme (0.377). More aggressive penalties do not lead to further improvements and, in some cases, degrade performance. Given the limited gains observed and the lack of substantial improvement in validation performance, class-weighted SVM variants are not considered further in the main results.

Table A.2: Validation-set ranking performance of the SVM model under different class-weight configurations.

Class weight	ROC-AUC	PR-AUC
{0:1, 1:50}	0.950	0.402
{0:1, 1:100}	0.953	0.391
`balanced`	0.955	0.377
{0:1, 1:172}	0.955	0.377
{0:1, 1:250}	0.956	0.361
{0:1, 1:350}	0.957	0.347

A.2 Code & Data

A ZIP file is provided containing the complete codebase together with the corresponding CSV data files.

B Statement of Authorship

”We hereby declare that this research project is our own work, that it has been created by us without the help of others, using only the sources referenced, and that we will not supply any copies of this paper to any third parties without written permission by the supervisor of this project”

At the same time, all rights to this project are hereby assigned to ZHAW Zurich University of Applied Sciences, except for the right to be identified as its author.

Place, Date:

Winterthur, 22.01.2026

Signatures:



Francis Du	Dominik Kaufmann	Max Heinzer